

□ □ □ □ □ □ □ □ □ □

— Anti-Interference-2D

1111

Anti-Interference-2D

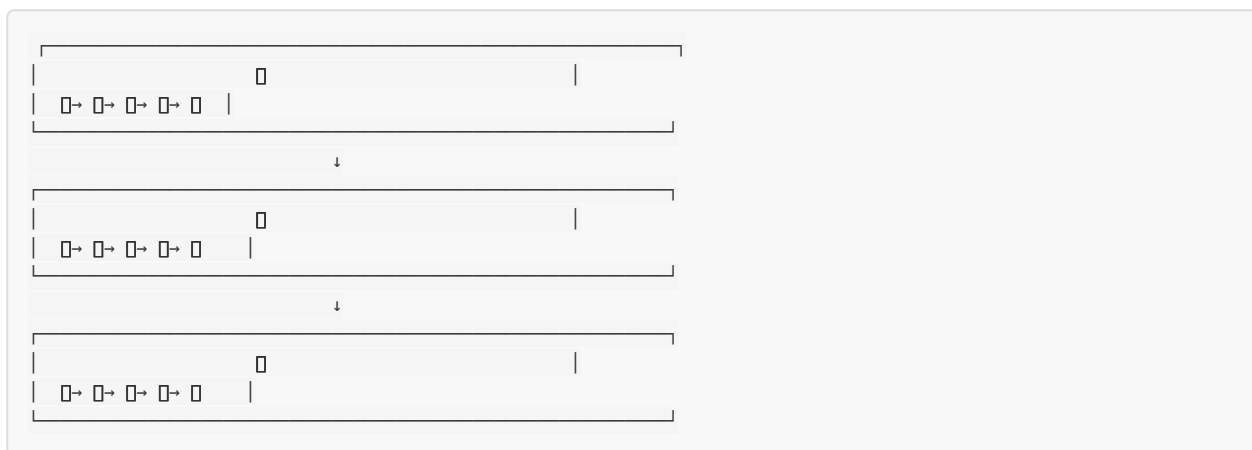
1. □□□□□□

1.1 関数型[51+][52+]

Grasp Planning

項目	内容	備考
材料費	鉄板、鉄釘、木材、漆料	
労務費	大工、漆工、足場工	
その他	足場、養生、仮設	
合計		

1.2 関数型プログラミング[52+]



1.3

	(x, y, z)	
	(α, β, γ)	X/Y/Z
	w	
	F	

$$Q(g, O) = w_1 \cdot S_{\text{reachability}} + w_2 \cdot S_{\text{stability}} + w_3 \cdot S_{\text{safety}} + w_4 \cdot S_{\text{efficiency}}$$

- $S_{\text{reachability}}$
- $S_{\text{stability}}$
- S_{safety}
- $S_{\text{efficiency}}$

2.

2.1

Anti-Interference-2D

```

class GraspPointSelector:
    """Anti-Interference-2D"""

    def __init__(self, config):
        self.min_edge_distance = config.get('min_edge_distance', 30) # mm
        self.max_edge_distance = config.get('max_edge_distance', 100) # mm
        self.defect_safe_margin = config.get('defect_safe_margin', 50) # mm

    def select_grasp_points(self, steel_contour, defect_regions, edge_quality_map):
        """
        Args:
            steel_contour: []
            defect_regions: [][{'bbox': [x1,y1,x2,y2], 'type': '[]'}, ...]
            edge_quality_map: []

        Returns:
            grasp_points: [][{'point': (x,y), 'angle': 0, 'quality': score}, ...]
        """
        candidates = []

        # []
        for i, point in enumerate(steel_contour):
            x, y = point

            # []
            edge_dist = self._compute_edge_distance(x, y, steel_contour)
            if not (self.min_edge_distance <= edge_dist <= self.max_edge_distance):
                continue

            # []
            if self._is_near_defect(x, y, defect_regions):
                continue

            # []
            angle = self._compute_grasp_angle(i, steel_contour)

            # []
            quality = self._evaluate_grasp_quality(
                x, y, angle, edge_quality_map, defect_regions, steel_contour)

            candidates.append({
                'point': (x, y),
                'angle': angle,
                'quality': quality,
                'edge_distance': edge_dist
            })

        # []
        candidates.sort(key=lambda c: c['quality'], reverse=True)

        return candidates[:10] # []op10

    def _compute_edge_distance(self, x, y, contour):
        """[]"""
        min_dist = float('inf')
        for px, py in contour:
            dist = np.sqrt((x - px)**2 + (y - py)**2)
            min_dist = min(min_dist, dist)
        return min_dist

    def _is_near_defect(self, x, y, defect_regions):
        """[]"""
        for defect in defect_regions:
            x1, y1, x2, y2 = defect['bbox']

```

```

        # []
        margin = self.defect_safe_margin
        if (x1-margin <= x <= x2+margin) and (y1-margin <= y <= y2+margin):
            return True
        return False

def _compute_grasp_angle(self, idx, contour):
    """[]"""
    n = len(contour)
    prev_idx = (idx - 1) % n
    next_idx = (idx + 1) % n

    dx = contour[next_idx][0] - contour[prev_idx][0]
    dy = contour[next_idx][1] - contour[prev_idx][1]

    # []
    angle = np.arctan2(dx, -dy) # []

    return angle

def _evaluate_grasp_quality(self, x, y, angle, edge_quality_map, defect_regions, contour):
    """[]"""
    quality = 1.0

    # []
    quality *= edge_quality_map.get((int(x), int(y)), 0.5)

    # []
    edge_dist = self._compute_edge_distance(x, y, contour)
    optimal_dist = (self.min_edge_distance + self.max_edge_distance) / 2
    distance_score = 1.0 - abs(edge_dist - optimal_dist) / (self.max_edge_distance - self.min_edge_distance)
    quality *= (0.5 + 0.5 * distance_score)

    # []
    for defect in defect_regions:
        dist_to_defect = self._distance_to_bbox(x, y, defect['bbox'])
        if dist_to_defect < 100:
            quality *= 0.5 # []

    # []
    curvature = self._compute_curvature(x, y, contour)
    quality *= (1.0 - 0.2 * curvature) # []

    return quality

def _distance_to_bbox(self, x, y, bbox):
    """[]"""
    x1, y1, x2, y2 = bbox
    dx = max(x1 - x, 0, x - x2)
    dy = max(y1 - y, 0, y - y2)
    return np.sqrt(dx**2 + dy**2)

def _compute_curvature(self, x, y, contour):
    """[]"""
    n = len(contour)
    idx = self._find_nearest_point(x, y, contour)
    if idx < 1 or idx >= n - 1:
        return 0

    # []
    p1 = contour[(idx - 1) % n]
    p2 = contour[idx]
    p3 = contour[(idx + 1) % n]

    # []
    v1 = np.array([p2[0] - p1[0], p2[1] - p1[1]])
    v2 = np.array([p3[0] - p2[0], p3[1] - p2[1]])

```

```

cos_angle = np.dot(v1, v2) / (np.linalg.norm(v1) * np.linalg.norm(v2) + 1e-6)
curvature = (1 - cos_angle) / 2 # [0,1]

return curvature

def _find_nearest_point(self, x, y, contour):
    """
    min_dist = float('inf')
    min_idx = 0
    for i, (px, py) in enumerate(contour):
        dist = (x - px)**2 + (y - py)**2
        if dist < min_dist:
            min_dist = dist
            min_idx = i
    return min_idx

```

2.2 51

項目	内容	単位
質量	0.001	kg
長さ	0.001	m
時間	0.001	s
電圧	0.001	V
電流	0.001	A

51

$$S_{\text{stability}} = \mu \cdot F_n \cdot \frac{d}{2} - W \cdot \frac{d}{2}$$

51

- μ
- F_n
- d
- W

2.3 53

53

```

class GraspQualityCNN(nn.Module):
    """GNN"""

    def __init__(self, num_classes=7):
        super().__init__()
        # ResNet-18
        from torchvision.models import resnet18
        backbone = resnet18(pretrained=True)

        #
        self.conv1 = nn.Conv2d(4, 64, kernel_size=7, stride=2, padding=3, bias=False) # RGB+Depth

        #
        self.backbone = nn.Sequential(
            backbone.bn1,
            backbone.relu,
            backbone.maxpool,
            backbone.layer1,
            backbone.layer2,
            backbone.layer3,
            backbone.layer4,
        )

        #
        self.grasp_head = nn.Sequential(
            nn.AdaptiveAvgPool2d((1, 1)),
            nn.Flatten(),
            nn.Linear(512, 256),
            nn.ReLU(),
            nn.Dropout(0.5),
            nn.Linear(256, 1), #
            nn.Sigmoid()
        )

    def forward(self, rgb, depth, grasp_pose):
        """
        Args:
            rgb: RGB (B, 3, H, W)
            depth: (B, 1, H, W)
            grasp_pose: (B, 6) [x, y, z, roll, pitch, yaw]

        Returns:
            quality: (B, 1)
        """
        # RGB
        x = torch.cat([rgb, depth], dim=1)
        x = self.conv1(x)

        #
        features = self.backbone(x)

        #
        pooled = F.adaptive_avg_pool2d(features, (1, 1))
        pooled = torch.flatten(pooled, 1)

        #
        pose_features = self.encode_pose(grasp_pose)
        combined = torch.cat([pooled, pose_features], dim=1)

        #
        quality = self.grasp_head(combined)

        return quality

    def encode_pose(self, pose):
        """

```

```
# MLP
return F.relu(self.pose_fc1(pose))
```

3. 实验结果

3.1 实验结果 [52][54]

(1) RRT (Rapidly-exploring Random Tree)

RRT 算法流程图如下：

开始

1. 初始化
2. x_{random}
3. x_{random} 与 x_{nearest}
4. x_{nearest} 与 x_{random} 的距离 d
5. x_{new} 与 x_{nearest} 的距离 d
6. $d < 5$

结束

```

import numpy as np
import random

class RRTNode:
    """RRT"""
    def __init__(self, state, parent=None):
        self.state = state # []
        self.parent = parent # []

class RRTPlanner:
    """RRT"""

    def __init__(self, workspace_bounds, step_size=20, max_iterations=5000):
        """
        Args:
            workspace_bounds: [[x_min, x_max], [y_min, y_max], [z_min, z_max]]
            step_size: (mm)
            max_iterations:
        """
        self.workspace_bounds = workspace_bounds
        self.step_size = step_size
        self.max_iterations = max_iterations
        self.nodes = []

    def plan(self, start, goal, obstacle_check_fn=None):
        """
        Args:
            start: [x, y, z]
            goal: [x, y, z]
            obstacle_check_fn: bool

        Returns:
            path: []None
        """
        # []
        start_node = RRTNode(np.array(start))
        self.nodes = [start_node]

        for _ in range(self.max_iterations):
            # []
            if random.random() < 0.1: # 10%
                random_state = np.array(goal)
            else:
                random_state = self._sample_random_state()

            # []
            nearest_node = self._find_nearest(random_state)

            # []
            new_node = self._steer(nearest_node, random_state)

            # []
            if obstacle_check_fn is None or not obstacle_check_fn(new_node.state):
                new_node.parent = nearest_node
                self.nodes.append(new_node)

            # []
            if np.linalg.norm(new_node.state - np.array(goal)) < self.step_size:
                return self._extract_path(new_node)

        return None # []

    def _sample_random_state(self):
        """[]"""

```



```

state = []
for low, high in self.workspace_bounds:
    state.append(random.uniform(low, high))
return np.array(state)

def _find_nearest(self, state):
    """
    min_dist = float('inf')
    nearest = None
    for node in self.nodes:
        dist = np.linalg.norm(node.state - state)
        if dist < min_dist:
            min_dist = dist
            nearest = node
    return nearest

def _steer(self, from_node, to_state):
    """
    direction = to_state - from_node.state
    distance = np.linalg.norm(direction)

    if distance <= self.step_size:
        new_state = to_state
    else:
        new_state = from_node.state + (direction / distance) * self.step_size

    return RRTNode(new_state)

def _extract_path(self, end_node):
    """
    path = []
    node = end_node
    while node is not None:
        path.append(node.state.tolist())
        node = node.parent
    path.reverse()
    return path

```

(2) RRT* (RRT*) [54†]

RRT RRT rewires*

RRT

```

| RRT | RRT* | |
|---|---|---|
| RRT | RRT* | RRT* |
| RRT | RRT* | RRT* |
| RRT | RRT* | RRT* |

```

RRT

```

class RRTStarPlanner(RRTPlanner):
    """RRT*"""

    def __init__(self, workspace_bounds, step_size=20, max_iterations=5000, rewiring_radius=50):
        super().__init__(workspace_bounds, step_size, max_iterations)
        self.rewiring_radius = rewiring_radius
        self.cost = {0: 0} #

    def plan(self, start, goal, obstacle_check_fn=None):
        """RRT*"""
        start_node = RRTNode(np.array(start))
        self.nodes = [start_node]
        self.cost = {0: 0}

        for i in range(self.max_iterations):
            #
            if random.random() < 0.1:
                random_state = np.array(goal)
            else:
                random_state = self._sample_random_state()

            #
            nearest_idx, nearest_node = self._find_nearest_with_idx(random_state)

            #
            new_node = self._steer(nearest_node, random_state)

            if obstacle_check_fn is not None and obstacle_check_fn(new_node.state):
                continue

            # RRT*
            new_node.parent, new_idx = self._find_best_parent(new_node, i)

            if new_node.parent is None:
                continue

            #
            self.cost[new_idx] = self.cost[id(new_node.parent)] + \
                np.linalg.norm(new_node.state - new_node.parent.state)

            #
            self.nodes.append(new_node)

            # Rewire
            self._rewire(new_node, new_idx, i)

            #
            if np.linalg.norm(new_node.state - np.array(goal)) < self.step_size:
                return self._extract_path(new_node)

        return None

    def _find_best_parent(self, new_node, new_node_idx):
        """
        best_parent = None
        best_cost = float('inf')
        best_idx = None

        for i, node in enumerate(self.nodes):
            dist = np.linalg.norm(new_node.state - node.state)
            if dist > self.step_size:
                continue

            if self._is_collision_free(node.state, new_node.state):
                cost = self.cost.get(i, float('inf')) + dist
                if cost < best_cost:

```

```

        best_cost = cost
        best_parent = node
        best_idx = i

    return best_parent, best_idx

def _rewire(self, new_node, new_idx, new_node_idx):
    """
    for i, node in enumerate(self.nodes):
        if i == new_idx:
            continue

        dist = np.linalg.norm(new_node.state - node.state)
        if dist > self.rewire_radius:
            continue

        if not self._is_collision_free(new_node.state, node.state):
            continue

        new_cost = self.cost[new_idx] + dist
        if new_cost < self.cost.get(i, float('inf')):
            #
            old_parent = node.parent
            node.parent = new_node
            self.cost[i] = new_cost

            #
            self._update_descendant_costs(node, old_parent)

def _update_descendant_costs(self, node, old_parent):
    """
    #
    pass

def _find_nearest_with_idx(self, state):
    """
    min_dist = float('inf')
    nearest = None
    nearest_idx = None

    for i, node in enumerate(self.nodes):
        dist = np.linalg.norm(node.state - state)
        if dist < min_dist:
            min_dist = dist
            nearest = node
            nearest_idx = i

    return nearest_idx, nearest

def _is_collision_free(self, state1, state2):
    """
    #
    num_checks = 10
    for t in np.linspace(0, 1, num_checks):
        point = (1 - t) * state1 + t * state2
        # TODO:
        # if is_in_obstacle(point):
        #     return False
    return True

```

(3) RRT-Connect[52†]

RRT

```

class RRTConnectPlanner:
    """RRT-Connect"""

    def __init__(self, workspace_bounds, step_size=20, max_iterations=5000):
        self.workspace_bounds = workspace_bounds
        self.step_size = step_size
        self.max_iterations = max_iterations

    def plan(self, start, goal, obstacle_check_fn=None):
        """RRT-Connect"""
        # []
        tree_start = [RRTNode(np.array(start))]
        tree_goal = [RRTNode(np.array(goal))]

        for _ in range(self.max_iterations):
            # []
            state = self._sample_random_state()
            nearest = self._find_nearest(tree_start, state)
            new_node = self._steer(nearest, state)

            if obstacle_check_fn is None or not obstacle_check_fn(new_node.state):
                new_node.parent = nearest
                tree_start.append(new_node)

            # []
            nearest_goal = self._find_nearest(tree_goal, new_node.state)
            if np.linalg.norm(new_node.state - nearest_goal.state) < self.step_size:
                # []
                return self._build_path(new_node, nearest_goal)

            # []
            state = self._sample_random_state()
            nearest = self._find_nearest(tree_goal, state)
            new_node = self._steer(nearest, state)

            if obstacle_check_fn is None or not obstacle_check_fn(new_node.state):
                new_node.parent = nearest
                tree_goal.append(new_node)

            # []
            nearest_start = self._find_nearest(tree_start, new_node.state)
            if np.linalg.norm(new_node.state - nearest_start.state) < self.step_size:
                # []
                return self._build_path(nearest_start, new_node)

        return None

    def _build_path(self, node_start, node_goal):
        """
        path1 = self._extract_path(node_start)
        path2 = self._extract_path(node_goal)
        path2.reverse()
        return path1 + path2
        """

    def _sample_random_state(self):
        """
        state = []
        for low, high in self.workspace_bounds:
            state.append(random.uniform(low, high))
        return np.array(state)
        """

    def _find_nearest(self, tree, state):
        """
        return min(tree, key=lambda n: np.linalg.norm(n.state - state))
        """

    def _steer(self, from_node, to_state):

```

```

    """
    direction = to_state - from_node.state
    distance = np.linalg.norm(direction)
    if distance <= self.step_size:
        new_state = to_state
    else:
        new_state = from_node.state + (direction / distance) * self.step_size
    return RRTNode(new_state)

def _extract_path(self, end_node):
    """
    path = []
    node = end_node
    while node is not None:
        path.append(node.state.tolist())
        node = node.parent
    path.reverse()
    return path

```

3.2 PRM (Probabilistic Roadmap)[54†]

```

class PRMPlanner:
    """PRM"""

    def __init__(self, workspace_bounds, num_samples=1000, connection_radius=50):
        """
        Args:
            workspace_bounds: []
            num_samples: []
            connection_radius: []
        """
        self.workspace_bounds = workspace_bounds
        self.num_samples = num_samples
        self.connection_radius = connection_radius
        self.graph = {} # {}

    def build_roadmap(self, obstacle_check_fn=None):
        """
        # []
        samples = []
        for _ in range(self.num_samples):
            sample = self._sample_state()
            if obstacle_check_fn is None or not obstacle_check_fn(sample):
                samples.append(sample)

        # []
        for i, sample in enumerate(samples):
            self.graph[i] = []
            for j, other in enumerate(samples):
                if i == j:
                    continue
                dist = np.linalg.norm(sample - other)
                if dist > self.connection_radius:
                    continue
                if obstacle_check_fn is not None:
                    if not self._is_edge_valid(sample, other, obstacle_check_fn):
                        continue
                self.graph[i].append((j, dist))

    def query(self, start, goal, obstacle_check_fn=None):
        """
        # []
        start_idx = len(self.graph)
        goal_idx = len(self.graph) + 1
        self.graph[start_idx] = []
        self.graph[goal_idx] = []

        # []
        for i in self.graph:
            if i >= start_idx:
                continue
            dist_start = np.linalg.norm(np.array(start) - np.array(list(self.graph.keys())[i]))
            if dist_start < self.connection_radius:
                self.graph[start_idx].append((i, dist_start))

        # []
        # ... []...

        # A*
        path = self._astar(start_idx, goal_idx)
        return path

    def _astar(self, start, goal):
        """A*"""
        import heapq

        open_set = [(0, start)]

```

```

came_from = {}
g_score = {start: 0}

while open_set:
    _, current = heapq.heappop(open_set)

    if current == goal:
        return self._reconstruct_path(came_from, current)

    for neighbor, dist in self.graph.get(current, []):
        tentative_g = g_score.get(current, float('inf')) + dist
        if tentative_g < g_score.get(neighbor, float('inf')):
            came_from[neighbor] = current
            g_score[neighbor] = tentative_g
            f_score = tentative_g + self._heuristic(neighbor, goal)
            heapq.heappush(open_set, (f_score, neighbor))

    return None

def _heuristic(self, a, b):
    """
    #
    return 0

def _reconstruct_path(self, came_from, current):
    """
    path = [current]
    while current in came_from:
        current = came_from[current]
        path.append(current)
    path.reverse()
    return path

def _sample_state(self):
    """
    state = []
    for low, high in self.workspace_bounds:
        state.append(random.uniform(low, high))
    return np.array(state)

def _is_edge_valid(self, p1, p2, check_fn):
    """
    for t in np.linspace(0, 1, 10):
        point = (1 - t) * p1 + t * p2
        if check_fn(point):
            return False
    return True

```

3.3 [52†][54†]

⚡	⚡	⚡	⚡	⚡
RRT	⚡	⚡	⚡	$O(n)$
RRT*	⚡	⚡	⚡	$O(n \log n)$
RRT-Connect	⚡	⚡	⚡	$O(n)$
PRM	⚡	⚡/⚡	⚡	$O(n^2)$
A*	⚡	⚡	⚡	$O(b^d)$

4. 平滑处理

4.1 平滑处理[52f]

RRT

(1) 平滑处理

```
def smooth_path_gradient(path, obstacles, iterations=100, step_size=0.1):
    """
    """
    smoothed = np.array(path, dtype=np.float64)

    for _ in range(iterations):
        for i in range(1, len(smoothed) - 1):
            # 
            smooth_grad = smoothed[i-1] + smoothed[i+1] - 2 * smoothed[i]

            # 
            obs_grad = compute_obstacle_gradient(smoothed[i], obstacles)

            # 
            smoothed[i] += step_size * (smooth_grad - 0.1 * obs_grad)

            # 
            if is_collision(smoothed[i], obstacles):
                smoothed[i] -= step_size * obs_grad

    return smoothed.tolist()

def compute_obstacle_gradient(point, obstacles, epsilon=1e-6):
    """
    """
    grad = np.zeros_like(point)
    for obs in obstacles:
        dist = np.linalg.norm(point - obs)
        if dist < 100: # 
            grad += (point - obs) / (dist + epsilon)
    return grad
```


(2) B-spline

```

from scipy.interpolate import splprep, splev
import numpy as np

def b_spline_smooth(path, num_points=100):
    """
    B-spline smoothing

    Args:
        path: list of [x, y, z], ...
        num_points: int

    Returns:
        smoothed: list
    """
    if len(path) < 4:
        return path

    path = np.array(path)

    # t
    t = np.linspace(0, 1, len(path))
    t_smooth = np.linspace(0, 1, num_points)

    # smoothed
    smoothed = np.zeros((num_points, path.shape[1]))
    for i in range(path.shape[1]):
        # tck
        k = 3 if len(path) > 3 else len(path) - 1
        tck, _ = splprep([path[:, i], t], s=0, k=k)
        smoothed[:, i] = splev(t_smooth, tck)[0]

    # smoothed
    smoothed[:, 2] = np.interp(t_smooth, t, path[:, 2])

    return smoothed.tolist()

```

(3) Bezier

```

def bezier_smoothing(path, segments=10):
    """
    Bezier smoothing

    smoothed = []
    for i in range(len(path) - 1):
        p0 = np.array(path[i])
        p1 = np.array(path[i + 1])

        # mid
        mid = (p0 + p1) / 2

        for t in range(segments + 1):
            t_norm = t / segments
            # (1-t)^2P0 + 2(1-t)tP1 + t^2P2
            point = (1 - t_norm)**2 * p0 + 2 * (1 - t_norm) * t_norm * mid + t_norm**2 * p1
            smoothed.append(point.tolist())

    return smoothed

```

4.2 平滑[54]

平滑处理

項目	単位
基本料	円/月
オプション料	円/月
その他	円/月
合計	円/月

□□□□□□□

-
- 
- S 

```
def quintic_polynomial_trajectory(q0, qf, v0, vf, a0, af, T):
    """
    []

    Args:
        q0, qf: []
        v0, vf: []
        a0, af: []
        T: []

    Returns:
        trajectory: []q(t)
    """
    # []
    #  $q(t) = a_0 + a_1 t + a_2 t^2 + a_3 t^3 + a_4 t^4 + a_5 t^5$ 

    a0_coef = q0
    a1_coef = v0
    a2_coef = a0 / 2

    # [a3, a4, a5]
    A = np.array([
        [T**3, T**4, T**5],
        [3*T**2, 4*T**3, 5*T**4],
        [6*T, 12*T**2, 20*T**3]
    ])
    b = np.array([qf - q0 - v0*T - a0*T**2/2, vf - v0 - a0*T, af - a0])

    a345 = np.linalg.solve(A, b)

    def trajectory(t):
        if t < 0:
            t = 0
        elif t > T:
            t = T

        return (a0_coef + a1_coef*t + a2_coef*t**2 +
                a345[0]*t**3 + a345[1]*t**4 + a345[2]*t**5)

    return trajectory
```

4.3 鋼鋼鋼鋼[61†]


```
def dynamic_replan(planner, current_path, obstacles, sensor_data):
    """
    Args:
        planner:
        current_path:
        obstacles:
        sensor_data:

    Returns:
        new_path:
    """
    #
    predicted_obstacles = predict_obstacle_motion(obstacles, sensor_data)

    # BRT*
    for obs in predicted_obstacles:
        planner.add_obstacle(obs)

    #
    start = current_path[-1] #
    goal = current_path[-1] #

    new_path = planner.plan(start, goal, obstacle_check_fn=lambda s: is_in_obstacle(s, predicted_obstacles))

    return new_path

def predict_obstacle_motion(obstacles, sensor_data):
    """
    predicted = []
    for obs, data in zip(obstacles, sensor_data):
        velocity = data.get('velocity', [0, 0, 0])
        predicted_pos = np.array(obs['position']) + np.array(velocity) * data.get('time_delta', 0.1)
        predicted.append({
            'position': predicted_pos,
            'shape': obs.get('shape', 'sphere'),
            'size': obs.get('size', [50, 50, 50])
        })
    return predicted
```

5. [54]

5.1

	w1	
	w2	
	w3	/jerk
	w4	
	w5	

$$\min_{\text{path}} J(\text{path}) = w_1 \cdot L(\text{path}) + w_2 \cdot S(\text{path})^{-1} + w_3 \cdot J(\text{path}) + w_4 \cdot V(\text{path}) + w_5 \cdot T(\text{path})$$

5.2 Pareto

Pareto

```

class MultiObjectiveOptimizer:
    """
    """

    def __init__(self, weights=None):
        self.weights = weights or {
            'length': 0.3,
            'safety': 0.3,
            'smoothness': 0.2,
            'time': 0.2
        }

    def optimize(self, paths):
        """
        """

        Returns:
            pareto_front: Pareto
        """
        evaluated = []
        for path in paths:
            metrics = self._evaluate_path(path)
            score = self._compute_score(metrics)
            evaluated.append((path, metrics, score))

        #
        pareto_front = self._non_dominated_sort(evaluated)

        return pareto_front

    def _evaluate_path(self, path):
        """
        """
        return {
            'length': self._compute_length(path),
            'safety': self._compute_safety(path),
            'smoothness': self._compute_smoothness(path),
            'time': self._compute_time(path)
        }

    def _compute_score(self, metrics):
        """
        """
        return (self.weights['length'] * metrics['length'] +
                self.weights['safety'] * metrics['safety'] +
                self.weights['smoothness'] * metrics['smoothness'] +
                self.weights['time'] * metrics['time'])

    def _non_dominated_sort(self, evaluated):
        """
        """
        pareto = []
        for i, (path_i, metrics_i, _) in enumerate(evaluated):
            is_dominated = False
            for j, (path_j, metrics_j, _) in enumerate(evaluated):
                if i == j:
                    continue
                if self._dominates(metrics_j, metrics_i):
                    is_dominated = True
                    break
            if not is_dominated:
                pareto.append(path_i)

        return pareto

    def _dominates(self, a, b):
        """
        """
        better_in_any = False
        for key in a:
            if a[key] > b[key]:

```

```
        better_in_any = True
    elif a[key] < b[key]:
        return False
    return better_in_any
```

6. [53†]

6.1

DRL

```

class GraspPolicyDDPG:
    """DDPG"""

    def __init__(self, state_dim, action_dim, action_bound):
        self.actor = nn.Sequential(
            nn.Linear(state_dim, 256),
            nn.ReLU(),
            nn.Linear(256, 256),
            nn.ReLU(),
            nn.Linear(256, action_dim),
            nn.Tanh() # 
        )

        self.critic = nn.Sequential(
            nn.Linear(state_dim + action_dim, 256),
            nn.ReLU(),
            nn.Linear(256, 256),
            nn.ReLU(),
            nn.Linear(256, 1) # Q
        )

        self.action_bound = action_bound
        self.target_actor = copy.deepcopy(self.actor)
        self.target_critic = copy.deepcopy(self.critic)

    def select_action(self, state, noise=True):
        """
        action = self.actor(state).detach().numpy()
        if noise:
            action += np.random.randn(*action.shape) * 0.1
        action = np.clip(action, -self.action_bound, self.action_bound)
        return action
        """

    def update(self, replay_buffer, batch_size=64, gamma=0.99, tau=0.005):
        """
        # 
        states, actions, rewards, next_states, dones = replay_buffer.sample(batch_size)

        # Q
        with torch.no_grad():
            next_actions = self.target_actor(next_states)
            target_q = self.target_critic(next_states, next_actions)
            target_q = rewards + gamma * (1 - dones) * target_q

        # Critic
        current_q = self.critic(states, actions)
        critic_loss = F.mse_loss(current_q, target_q)
        self.critic_optimizer.zero_grad()
        critic_loss.backward()
        self.critic_optimizer.step()

        # Actor
        actions_pred = self.actor(states)
        actor_loss = -self.critic(states, actions_pred).mean()
        self.actor_optimizer.zero_grad()
        actor_loss.backward()
        self.actor_optimizer.step()

        # 
        self._soft_update(self.target_actor, self.actor, tau)
        self._soft_update(self.target_critic, self.critic, tau)

    def _soft_update(self, target, source, tau):
        """
        for target_param, param in zip(target.parameters(), source.parameters()):
            target_param.data.copy_(tau * param.data + (1 - tau) * target_param.data)
        """

```

□□□□□□

$$r = r_{\text{success}} + w_1 \cdot r_{\text{stability}} + w_2 \cdot r_{\text{time}} + w_3 \cdot r_{\text{safety}}$$

□□□	□□	□
r_{success}	□□□	+10
$r_{\text{stability}}$	□□□□	+1 * stability_score
r_{time}	□□□	-0.01 * time
r_{safety}	□□□	-1□□□□□

6.2 □□□□[53†]

□□□□□□□□□□


```

def behavioral_cloning(expert_demonstrations, state_dim, action_dim):
    """
    Args:
        expert_demonstrations: [(state, action), ...]

    Returns:
        policy:
    """
    states, actions = zip(*expert_demonstrations)
    states = torch.FloatTensor(states)
    actions = torch.FloatTensor(actions)

    policy = nn.Sequential(
        nn.Linear(state_dim, 256),
        nn.ReLU(),
        nn.Linear(256, 256),
        nn.ReLU(),
        nn.Linear(256, action_dim),
        nn.Tanh()
    )

    optimizer = torch.optim.Adam(policy.parameters(), lr=1e-3)
    loss_fn = nn.MSELoss()

    for epoch in range(100):
        optimizer.zero_grad()
        pred_actions = policy(states)
        loss = loss_fn(pred_actions, actions)
        loss.backward()
        optimizer.step()

    return policy

def dagger_update(policy, expert_policy, dataset, num_iterations=10):
    """
    DAgger
    Args:
        policy:
        expert_policy:
        dataset:
    """
    for _ in range(num_iterations):
        #
        new_data = collect_rollouts(policy, env)

        #
        for state in new_data:
            expert_action = expert_policy(state)
            dataset.append((state, expert_action))

        #
        policy = behavioral_cloning(dataset)

```

6.3

7.

7.1

1. |

↓

2. |

↓

3. Z |

↓

4. |

↓

5. |

↓

6. |

7.2

```

def force_position_control(current_force, target_force, position, target_position,
                           kp_f=0.5, kp_p=0.1):
    """
    Args:
        current_force: float
        target_force: float
        position: float
        target_position: float
        kp_f: float
        kp_p: float

    Returns:
        output: float
    """
    # force error
    force_error = target_force - current_force
    force_adjustment = kp_f * force_error

    # position error
    position_error = target_position - position
    position_adjustment = kp_p * position_error

    # total adjustment
    output = position_adjustment + force_adjustment
    return output

def impedance_control(desired_force, actual_force, actual_position,
                      M=1.0, B=10.0, K=100.0):
    """
    Args:
        desired_force: float
        actual_force: float
        actual_position: float
        M: float
        B: float
        K: float

    Returns:
        position_adjustment: float
    """
    force_error = desired_force - actual_force
    #  $M \cdot a + B \cdot v + K \cdot x = F$ 
    # position adjustment
    position_adjustment = force_error / K
    return position_adjustment

```

7.3 阻抗控制[51†]

参数	单位	范围
质量	kg	$m < \infty$
阻尼	N·s/m	$b \geq 0$
刚度	N/m	$k > 0$
位置	m	$x \in \mathbb{R}$

```

def grasp_stability_check(force_readings, torque_readings, expected_force):
    """
    Returns:
        is_stable: bool
        stability_score: [0, 1]
    """
    # 
    max_torque = max(abs(t) for t in torque_readings)
    if max_torque > TORQUE_THRESHOLD:
        return False, 0.0

    # 
    force_error = abs(sum(force_readings) - expected_force) / expected_force
    if force_error > FORCE_ERROR_THRESHOLD:
        return False, 0.0

    # 
    stability_score = 1.0 - min(1.0, force_error / FORCE_ERROR_THRESHOLD)
    return True, stability_score

def adaptive_force_control(current_force, target_force, slip_detected):
    """
    Args:
        current_force: 
        target_force: 
        slip_detected: 

    Returns:
        new_target_force: 
    """
    if slip_detected:
        # 
        new_target_force = target_force * 1.2
        new_target_force = min(new_target_force, MAX_SAFE_FORCE)
    else:
        # 
        new_target_force = target_force * 0.98

    return new_target_force

```

8. [51+][52+]

8.1

		Anti-Interference-2D
	6	6
	$\geq 20\text{kg}$	30kg+
	$\geq 1500\text{mm}$	2000mm
	$\leq \pm 0.1\text{mm}$	$\pm 0.05\text{mm}$
	$\geq 1\text{m/s}$	1.5m/s

8.2

	50-200mm
	500-2000N
	≤±0.05mm

8.3

	6	
	12MP	
	ToF	

8.4 配置

```
# Anti-Interference-2D
# grasp_config.yaml

grasp_planning:
  #
  grasp_point:
    min_distance_from_edge: 30      # mm
    max_distance_from_edge: 100     # mm
    defect_safe_margin: 50          # mm
    quality_threshold: 0.7
    max_grasp_angle: 15             #

  #
  path_planning:
    algorithm: "RRT*"               # RRT / RRT* / PRM
    max_iterations: 5000
    step_size: 20                   # mm
    goal_bias: 0.1
    rewiring_radius: 50             # mm (RRT*)

  #
  smoothing:
    algorithm: "b_spline"           # bezier / b_spline / gradient
    iterations: 100
    smoothness_weight: 0.3

  #
  motion:
    max_velocity: 500               # mm/s
    max_acceleration: 2000          # mm/s²
    max_jerk: 10000                 # mm/s³
    target_force: 300               # N

  #
  collision_avoidance:
    enabled: True
    safety_margin: 30               # mm
    dynamic_replan: True
```

9. 规划库[52†][54†]

9.1 规划库

	语言	特点	GitHub
OMPL	C++/Python	通用规划库	kavrakilab/ompl
MoveIt	C++/Python	ROS规划库	ros-planning/moveit
SBPL	C++	快速规划	sbpl/sbpl
OpenRAVE	C++/Python	机器人仿真与规划	openrave/openrave
PyBullet	Python	物理引擎+规划	google/pybullet

9.2

		GitHub
Graspit!		graspit-simulator/graspit
Dex-Net		dvgup/GraspNet
Contact-GraspNet		robotology/contact_graspnet

9.3 OMPL

```

from ompl import geometric as og
from ompl import base as ob

def plan_with_ompl(start, goal, bounds, obstacle_fn):
    """OMPL"""

    #
    space = ob.RealVectorStateSpace(len(start))
    bounds_obj = ob.RealVectorBounds(len(start))
    for i, (low, high) in enumerate(bounds):
        bounds_obj.setLow(i, low)
        bounds_obj.setHigh(i, high)
    space.setBounds(bounds_obj)

    #
    si = ob.SpaceInformation(space)
    si.setStateValidityChecker(ob.StateValidityCheckerFn(obstacle_fn))

    start_state = ob.State(space)
    goal_state = ob.State(space)
    for i, val in enumerate(start):
        start_state[i] = val
    for i, val in enumerate(goal):
        goal_state[i] = val

    pdef = ob.ProblemDefinition(si)
    pdef.setStartAndGoalStates(start_state, goal_state)

    #
    planner = og.RRTstar(si)
    planner.setProblemDefinition(pdef)

    #
    planner.solve(5.0) # 5

    if pdef.hasApproximateSolution():
        path = pdef.getSolutionPath()
        return path.getStates()
    return None

```

10. [40+][62+]

10.1

	3D+	$\pm 2\text{mm}$
	YOLOv8	$> 95\%$
	RRT*+	$< 0.5\text{s}$

10.2 [62+]

- 3D
-
-

11. Anti-Interference-2D□□□□□□

11.1 □□□

```

class SteelGraspSystem:
    """
    Anti-Interference-2D
    """

    def __init__(self, config):
        #
        self.edge_detector = SteelEdgeDetector()
        self.defect_detector = DefectDetector()

        #
        self.grasp_selector = GraspPointSelector(config['grasp_point'])
        self.path_planner = RRTStarPlanner(
            config['path_planning']['workspace_bounds'],
            step_size=config['path_planning']['step_size'],
            max_iterations=config['path_planning']['max_iterations']
        )
        self.path_smoother = PathSmoother()

        #
        self.robot_controller = RobotController()
        self.force_controller = ForceController()

    def execute_grasp(self, rgb_image, depth_image, polarization_info=None):
        """
        Args:
            rgb_image: RGB
            depth_image:
            polarization_info:

        Returns:
            success:
            result:
        """
        # 1.
        edges, contours, _ = self.edge_detector.detect(rgb_image, polarization_info)
        defects = self.defect_detector.detect(rgb_image)

        # 2.
        grasp_points = self.grasp_selector.select_grasp_points(
            contours[0], defects, edges)

        if not grasp_points:
            return False, ""

        best_grasp = grasp_points[0]

        # 3.
        current_pos = self.robot_controller.get_current_position()
        goal_pos = best_grasp['point'] + [best_grasp.get('height', 100)]

        path = self.path_planner.plan(
            current_pos, goal_pos,
            obstacle_check_fn=self._check_collision
        )

        if path is None:
            return False, ""

        # 4.
        smoothed_path = self.path_smoother.b_spline_smooth(path)

        # 5.
        self.robot_controller.execute_path(smoothed_path)

```

```

# 6.
target_force = self.config['motion']['target_force']
self.force_controller.grasp(target_force)

# 7.
is_stable, score = self.force_controller.check_stability()
if not is_stable:
    self.robot_controller.release()
    return False, f"(score={score})"

return True, {
    'grasp_point': best_grasp,
    'stability_score': score,
    'path_length': len(smoothed_path)
}

```

12.

[51†]
 [52†] RRT/RRT*
 [54†] PRM
 [61†]
 [53†]

v1.0
 2026-04-27
 Anti-Interference-2D 2D
 2D